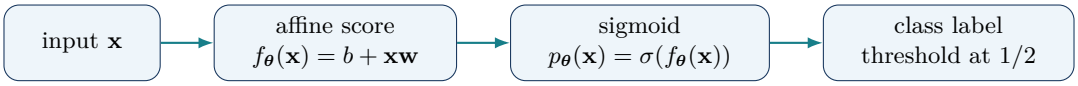
Week 3 Study Guide

Linear Classification

Foundations of Artificial Intelligence - SEMTM0051

What this lecture is about. Week 3 moves from **regression** (predicting a number) to **classification** (predicting a class label). The lecture develops binary logistic classification, explains why we train with a smooth logistic loss instead of the raw mistake count, derives the gradient used by gradient descent, and then extends the model to more than two classes using softmax and cross-entropy.

The main idea



For binary classification, the model first predicts a **probability of class 1**. A final class label is obtained by thresholding that probability.

Binary pipeline

$$\mathbf{x} \longrightarrow s = f_{\boldsymbol{\theta}}(\mathbf{x}) \longrightarrow p = \sigma(s) \longrightarrow h(\mathbf{x}) = \begin{cases} 0, & p < \frac{1}{2} \\ 1, & p \geq \frac{1}{2}. \end{cases}$$

Regression vs classification

	Regression	Classification
Output	Numerical value	Class label
Typical model output	$\widehat{y} \in \mathbb{R}$	Probability / class
Evaluation	Numerical error	Number of wrong labels
Lecture 3 fitting loss	–	Logistic loss (binary) / cross-entropy (multiclass)

Important distinction: the quantity we *train* may differ from the quantity we ultimately *evaluate*. In this lecture, the model is fitted with a smooth probability loss, while classification performance is evaluated by counting mistakes.

Notation and variables - what each symbol means

Symbol	Type	Meaning
N	positive integer	Number of observations in the dataset.
i	$1, \dots, N$	Index of an observation.
d	positive integer	Number of input features.
$\mathbf{x}_i = (x_{i1}, \dots, x_{id})$	$\mathbb{R}^{1\times d}$	Input-feature row for observation i.
x_{ij}	real number	Feature j of observation i.
y_i	$\{0, 1\}$ (binary)	Observed class label for observation i.
b	$\mathbb{R}$	Intercept / bias. Adjustable model parameter.
$\mathbf{w} = (w_1, \dots, w_d)^\top$	$\mathbb{R}^{d\times 1}$	Weight vector; one adjustable weight per feature.
$\boldsymbol{\theta}$	$\mathbb{R}^{(d+1)\times 1}$	All binary-model parameters collected together: $\boldsymbol{\theta} = (b, \mathbf{w}^\top)^\top$.
$f_{\boldsymbol{\theta}}(\mathbf{x})$	real number	Affine **score** or **logit**. It can be any real number.
$\sigma(s)$	$(0, 1)$	Sigmoid; converts a real score into a probability.
$p_{\boldsymbol{\theta}}(\mathbf{x})$	$(0, 1)$	Predicted probability that the input belongs to class 1.
$h_{p_{\boldsymbol{\theta}}}(\mathbf{x})$	$\{0, 1\}$	Final classifier label after thresholding the probability.
$\ell_{0,1}$	$\{0, 1\}$	Zero-one loss for one prediction: 0 if correct, 1 if wrong.
$\mathcal{E}(\boldsymbol{\theta})$	integer	Total number of classification mistakes over the dataset.
$\ell_{\log}$	nonnegative real	Logistic loss for one observation.
$\mathcal{L}_{\log}(\boldsymbol{\theta})$	nonnegative real	Sum of logistic losses over all observations.
η	positive real	Learning rate / step size in gradient descent.
t	$0, 1, 2, \dots$	Gradient-descent iteration number.

Parameters versus data

During fitting, the observed data $\mathbf{x}_i$ and y_i are **fixed**. The quantities b, $\mathbf{w}$, and therefore $\boldsymbol{\theta}$, are the **adjustable parameters** that the algorithm changes.

The binary affine score

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = b + \mathbf{x}\mathbf{w} = b + \sum_{j=1}^{d} x_j w_j.$$

Examples:

$$\begin{aligned} d = 1: &\quad b + x_1 w_1, \\ d = 2: &\quad b + x_1 w_1 + x_2 w_2, \\ d = 3: &\quad b + x_1 w_1 + x_2 w_2 + x_3 w_3. \end{aligned}$$

Binary classification: from a score to a decision

1. The sigmoid function

$$\boxed{\sigma(s) = \frac{1}{1+e^{-s}} = \frac{e^{s}}{1+e^{s}}}$$

It is smooth, increasing, and satisfies

$$0 < \sigma(s) < 1, \qquad \sigma(0) = \frac{1}{2}.$$

Therefore negative scores favour class 0, while positive scores favour class 1.

2. Probability model

$$\boxed{p_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(f_{\boldsymbol{\theta}}(\mathbf{x})) = \sigma(b + \mathbf{x}\mathbf{w})}$$

Interpretation:

$$p_{\boldsymbol{\theta}}(\mathbf{x}) \approx P(y = 1 \mid \mathbf{x}), \qquad 1 - p_{\boldsymbol{\theta}}(\mathbf{x}) \approx P(y = 0 \mid \mathbf{x}).$$

3. Plug-in classification rule

$$\boxed{h_{p_{\boldsymbol{\theta}}}(\mathbf{x}) = \begin{cases} 0, & p_{\boldsymbol{\theta}}(\mathbf{x}) < \frac{1}{2}, \\ 1, & p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \frac{1}{2}. \end{cases}}$$

Because the sigmoid is increasing and $\sigma(0) = 1/2$,

$$h_{p_{\boldsymbol{\theta}}}(\mathbf{x}) = 1 \iff p_{\boldsymbol{\theta}}(\mathbf{x}) \geq \frac{1}{2} \iff f_{\boldsymbol{\theta}}(\mathbf{x}) \geq 0 \iff b + \mathbf{x}\mathbf{w} \geq 0.$$

4. Decision boundary

The separating boundary is therefore

$$\boxed{b + \mathbf{x}\mathbf{w} = 0.}$$

$d = 1$	The boundary is a point on the real line.
$d = 2$	The boundary is a straight line in the plane.
$d = 3$	The boundary is a plane in three-dimensional space.
General d	The boundary is a hyperplane, provided $\mathbf{w} \neq \mathbf{0}$.

Changing b shifts the boundary. Changing the weights w_j changes how each feature contributes and can rotate/reorient the boundary.

Example: two features

For $d = 2$,

$$b + w_1 x_1 + w_2 x_2 = 0.$$

If $w_2 \neq 0$, this can be rearranged as

$$x_2 = -\frac{b}{w_2} - \frac{w_1}{w_2} x_1,$$

which is the familiar equation of a straight line.

Classification error and why we do not optimise it directly

Zero-one loss: one prediction

$$\ell_{0,1}(\widehat{y}, y) = \begin{cases} 0, & \widehat{y} = y, \\ 1, & \widehat{y} \neq y. \end{cases}$$

Every mistake receives the same penalty, regardless of how uncertain or confident the model was.

Mistake count: the whole dataset

$$\boxed{\mathcal{E}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{0,1}(h_{p_{\boldsymbol{\theta}}}(\mathbf{x}_i), y_i)}$$

This is exactly the number of misclassified observations.

Why gradient descent fails on the raw mistake count. When the decision boundary moves a little but does not cross any observed point, all predicted labels remain unchanged. Therefore $\mathcal{E}(\boldsymbol{\theta})$ is locally flat and its gradient is zero in those regions. Gradient descent then has no useful direction in which to move.

What the mistake count also ignores

Suppose the true class is 0.

	Prediction A	Prediction B
Predicted $P(y = 1)$	0.51	0.90
Predicted class	1	1
Zero-one loss	1	1
Probability of observed class 0	0.49	0.10

The first prediction is only slightly wrong; the second is confidently wrong. Zero-one loss treats them identically, but a probability-fitting loss should penalise the second much more.

Logistic loss (binary cross-entropy)

For a predicted probability p of class 1 and an observed label $y \in \{0, 1\}$,

$$\boxed{\ell_{\log}(p, y) = \begin{cases} -\log p, & y = 1, \\ -\log(1 - p), & y = 0, \end{cases}}$$

or equivalently

$$\boxed{\ell_{\log}(p, y) = -y \log p - (1 - y) \log(1 - p).}$$

Examples from the lecture:

$$\ell_{\log}(0.51, 0) = -\log(0.49) \approx 0.713,$$
$$\ell_{\log}(0.90, 0) = -\log(0.10) \approx 2.303.$$

Empirical logistic loss

For observation i,

$$p_i = p_{\boldsymbol{\theta}}(\mathbf{x}_i) = \sigma(b + \mathbf{x}_i \mathbf{w}).$$

Add the individual logistic losses:

$$\boxed{\mathcal{L}_{\log}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)\,.}$$

We **fit** the smooth objective $\mathcal{L}_{\log}$, while the actual classification measure remains the mistake count $\mathcal{E}(\boldsymbol{\theta})$.

Relation to mistakes

The lecture shows that for one observation,

$$\ell_{0,1}(\text{step}(s), y) \le \frac{\ell_{\log}(\sigma(s), y)}{\log 2}.$$

Summing over the dataset gives

$$\boxed{\mathcal{E}(\boldsymbol{\theta}) \le \frac{1}{\log 2}\,\mathcal{L}_{\log}(\boldsymbol{\theta}).}$$

So controlling the logistic loss gives a bound on the number of mistakes.

This does **not** mean the two quantities always move together at every gradient-descent step. The lecture explicitly shows a step where logistic loss decreases while the mistake count increases. Logistic loss is a smooth *surrogate* objective, not the same function as the mistake count.

Design matrix

As in Week 2, a leading column of ones includes the intercept:

$$\mathbf{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{N1} & \cdots & x_{Nd} \end{pmatrix} \in \mathbb{R}^{N \times (d+1)},$$

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} \in \mathbb{R}^{(d+1)\times 1}.$$

The first coordinate of $\boldsymbol{\theta}$ is the intercept; the remaining coordinates are the weights.

Vector of probabilities

Define

$$\mathbf{p}_{\boldsymbol{\theta}} = \begin{pmatrix} p_{\boldsymbol{\theta}}(\mathbf{x}_1) \\ \vdots \\ p_{\boldsymbol{\theta}}(\mathbf{x}_N) \end{pmatrix}, \qquad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix}.$$

Applying the sigmoid entrywise,

$$\boxed{\mathbf{p}_{\boldsymbol{\theta}} = \boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}).}$$

Deriving the logistic-loss gradient

The lecture breaks the loss for one observation into three functions:

$$\boldsymbol{\theta} \xrightarrow{F_i} \text{score} \xrightarrow{\sigma} \text{probability} \xrightarrow{G_i} \text{loss}.$$

1. Affine score

$$F_i(\boldsymbol{\theta}) = f_{\boldsymbol{\theta}}(\mathbf{x}_i) = (1, \mathbf{x}_i)\boldsymbol{\theta} = \sum_{r=1}^{d+1} X_{ir}\theta_r.$$

Therefore

$$\boxed{\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j} = X_{ij}.}$$

2. Sigmoid derivative

$$\boxed{\sigma'(s) = \sigma(s)\big(1 - \sigma(s)\big).}$$

Thus at observation i,

$$\sigma'(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) = p_i(1 - p_i).$$

3. Logistic loss derivative with respect to probability

Starting from

$$\ell_{\log}(p, y) = -y \log p - (1 - y) \log(1 - p),$$

the derivative is

$$\boxed{\frac{\partial \ell_{\log}}{\partial p}(p, y) = \frac{p - y}{p(1 - p)}.}$$

4. Chain rule

Multiply the three pieces:

$$\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \frac{p_i - y_i}{p_i(1 - p_i)} \cdot p_i(1 - p_i) \cdot X_{ij}.$$

The probability terms cancel:

$$\boxed{\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = (p_i - y_i)X_{ij}.}$$

Key pattern: for each observation, the gradient contribution is essentially

$$\text{prediction error in probability} \times \text{input feature}.$$

Here the probability error is $p_i - y_i$.

Empirical gradient and gradient descent

Because the empirical loss is a sum, the observation derivatives add:

$$\frac{\partial \mathcal{L}_{\log}}{\partial \theta_j}(\boldsymbol{\theta}) = \sum_{i=1}^{N} X_{ij}\big(p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i\big).$$

In matrix form,

$$\boxed{\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}) = \mathbf{X}^\top(\mathbf{p}_{\boldsymbol{\theta}} - \mathbf{y}) = \mathbf{X}^\top\left(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}\right).}$$

Gradient-descent update

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\, \nabla \mathcal{L}_{\log}\Big(\boldsymbol{\theta}^{(t)}\Big)}$$

or

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\, \mathbf{X}^\top\Big(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}^{(t)}) - \mathbf{y}\Big).}$$

$\boldsymbol{\theta}^{(t)}$	Current parameters at iteration t.
$\nabla \mathcal{L}_{\log}$	Direction of steepest local increase of logistic loss.
$-\nabla \mathcal{L}_{\log}$	Direction used to reduce the loss.
η	Learning rate controlling how far each update moves.

What the lecture's experiment shows

With the same initial coefficients, the lecture compares two learning rates:

	$\eta = 0.001$	$\eta = 0.002$
Initial logistic loss	51.048	51.048
Loss after first update	42.733	64.556
Behaviour	decreases	increases

Learning rate matters. A step that is too large can overshoot and increase the loss. The lecture's $\eta = 0.002$ run oscillates and performs much worse than the smaller-step run.

Another subtlety

The fitted logistic loss can improve even when the number of mistakes temporarily gets worse. Therefore, when reading optimisation graphs, distinguish carefully between:

- the **smooth training objective** $\mathcal{L}_{\log}$, and
- the **classification metric** $\mathcal{E}$.

More than two classes

Suppose there are K possible classes. For example, handwritten-digit recognition has ten classes: $0, 1, \ldots, 9$.

An input is still represented by a feature vector

$$\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^{1\times d},$$

where, for an image, the features can be pixel intensities read in a fixed order.

Probability vector

The model predicts one probability per class:

$$\mathbf{p}(\mathbf{x}) = \big(p_1(\mathbf{x}), \ldots, p_K(\mathbf{x})\big),$$

with

$$p_k(\mathbf{x}) > 0, \qquad \sum_{k=1}^{K} p_k(\mathbf{x}) = 1.$$

Multiclass plug-in rule

Choose the class with the largest predicted probability:

$$\boxed{h_{\mathbf{p}}(\mathbf{x}) = \max\left(\arg\max_{k\in\{1,\ldots,K\}} p_k(\mathbf{x})\right).}$$

The outer max is the lecture's tie-breaking convention: if several classes tie, choose the largest class index.

One affine score per class

For each class k,

$$f_{\boldsymbol{\theta}_k}(\mathbf{x}) = b_k + \mathbf{x}\mathbf{w}_k = (1, \mathbf{x})\boldsymbol{\theta}_k.$$

Collect the parameters in columns:

$$\boldsymbol{\Theta} = \begin{pmatrix} \boldsymbol{\theta}_1 & \cdots & \boldsymbol{\theta}_K \end{pmatrix} \in \mathbb{R}^{(d+1)\times K}.$$

The row of all class scores is

$$\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}) = (1, \mathbf{x})\boldsymbol{\Theta} \in \mathbb{R}^{1\times K}.$$

Softmax

For scores $\mathbf{s} = (s_1, \ldots, s_K)$,

$$\boxed{[\mathrm{softmax}(\mathbf{s})]_k = \frac{e^{s_k}}{\sum_{r=1}^{K} e^{s_r}}, \qquad k = 1, \ldots, K.}$$

Every component is positive and the components sum to one.

> Do **not** apply separate sigmoids to each class and assume the outputs form a probability distribution. Separate sigmoid outputs are not constrained to sum to one; softmax is designed to enforce that normalisation.

Multiclass probability model and cross-entropy

Apply softmax to the class scores:

$$\boxed{\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x}) = \operatorname{softmax}(\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}))\,.}$$

For class k,

$$\boxed{p_{\boldsymbol{\Theta},k}(\mathbf{x}) = \frac{\exp(f_{\boldsymbol{\theta}_k}(\mathbf{x}))}{\sum_{r=1}^{K} \exp(f_{\boldsymbol{\theta}_r}(\mathbf{x}))}.}$$

The largest score and the largest softmax probability identify the same class.

Lecture example with three classes

If

$$\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}) = (0, \log 2, \log 3),$$

then exponentiating gives

$$(e^0, e^{\log 2}, e^{\log 3}) = (1, 2, 3).$$

Normalising:

$$\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x}) = \left(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}\right),$$

so class 3 is predicted.

Cross-entropy loss for one observation

If the observed class is $y \in \{1, \ldots, K\}$,

$$\boxed{\ell_{\text{multi}}(\mathbf{p}, y) = -\log p_y.}$$

Only the probability assigned to the **observed class** enters the loss.

For the lecture example,

$$\mathbf{p} = \left(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}\right), \qquad y = 2,$$

so

$$\ell_{\text{multi}}(\mathbf{p}, 2) = -\log\left(\frac{1}{3}\right) = \log 3.$$

Multiclass empirical objective

For N labelled observations,

$$\boxed{\mathcal{L}_{\text{multi}}(\boldsymbol{\Theta}) = \sum_{i=1}^{N} \ell_{\text{multi}}(\mathbf{p}_{\boldsymbol{\Theta}}(\mathbf{x}_i), y_i)\,.}$$

The parameter matrix $\boldsymbol{\Theta}$ is fitted by minimising this summed loss, for example with gradient descent.

Final comparison

Task	Model prediction	Fitting loss
Regression	Numerical values	Squared loss
Binary classification	Sigmoid probabilities	Logistic loss
Multiclass classification	Softmax probabilities	Cross-entropy loss

Core message of Week 3: start with linear/affine scores, convert them to probabilities (sigmoid for two classes, softmax for many classes), fit those probabilities using a smooth loss, and evaluate the final classifier using class labels and mistake counts.

One-page revision sheet

Binary model

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = b + \mathbf{x}\mathbf{w}, \qquad p_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma(f_{\boldsymbol{\theta}}(\mathbf{x})), \qquad \sigma(s) = \frac{1}{1+e^{-s}}.$$

$$h(\mathbf{x}) = \begin{cases} 0, & p(\mathbf{x}) < 1/2 \\ 1, & p(\mathbf{x}) \geq 1/2 \end{cases} \qquad \Longleftrightarrow \qquad b + \mathbf{x}\mathbf{w} \gtreqless 0.$$

Binary losses

$$\ell_{0,1}(\widehat{y}, y) = \mathbf{1}\{\widehat{y} \neq y\}, \qquad \ell_{\log}(p, y) = -y \log p - (1-y)\log(1-p).$$

$$\mathcal{L}_{\log}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i).$$

Binary gradient

$$\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}) = \mathbf{X}^{\top}\big(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}\big).$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}_{\log}(\boldsymbol{\theta}^{(t)}).$$

Remember:

- Zero-one error is the evaluation measure but is not a useful smooth training objective.
- Logistic loss is a smooth surrogate; lower logistic loss does not guarantee fewer mistakes at every single step.
- Too large a learning rate can increase the loss.

Multiclass model

$$\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}) = (1, \mathbf{x})\boldsymbol{\Theta}, \qquad p_k(\mathbf{x}) = \frac{e^{f_k(\mathbf{x})}}{\sum_{r=1}^{K} e^{f_r(\mathbf{x})}}.$$

$$h(\mathbf{x}) = \arg\max_k p_k(\mathbf{x}), \qquad \ell_{\text{multi}}(\mathbf{p}, y) = -\log p_y.$$

Minimum notation to recognise quickly

$\mathbf{x}_i$	input row	y_i	observed class
b	bias/intercept	$\mathbf{w}$	feature weights
$\boldsymbol{\theta}$	binary parameter vector	$\boldsymbol{\Theta}$	multiclass parameter matrix
f	score/logit	p	probability
σ	sigmoid	**softmax**	multiclass normalisation
$\mathcal{E}$	mistake count	$\mathcal{L}$	training loss
η	learning rate	t	iteration

Prepared from Week 3 - Lecture 3: Linear Classification.